

RESTful API

What is an API?

- An API is short for an Application Programming Interface
- An API is a set of routines, functions, and tools that are available to a developer

Example of an API

- This is Twitter's API for tweets:
<https://developer.twitter.com/en/docs/tweets/data-dictionary/overview/tweet-object>
- A lot of online blogs and dating apps require you to login using your Facebook
- What is happening behind the scenes after you “authenticate”
 - The app will make a call to Facebook's graph API and pull back interests, liked pages, friends, name, birthday, etc.
 - A lot of these apps and blogs must use API because it streamlines “account creation”

Example of an API (cont.)

- A big one is career building websites with LinkedIn
 - Indeed
 - Monster
 - AngelList
- When you apply for a job and it autofills job history, name, and education
 - That is all serviced from LinkedIn's API

In Short

- MuleSoft explains it very well

<https://www.youtube.com/watch?v=s7wmiS2mSXY>

RESTful API

- They are all of the API examples that were discussed
- Representational state transfer
- All that means is that GET and POST requests are done via HTTP
 - Easily parsable by any language
- Just know for this class we will be dealing with REST APIs
 - Not SOAP - Simple Object Access Protocol (which no one uses anymore)
 - Not RPC = Remote Procedure Call (which no one uses anymore)
 - I don't know much about these types of APIs and neither should anyone in 2017...

API Visual



snoopdogg



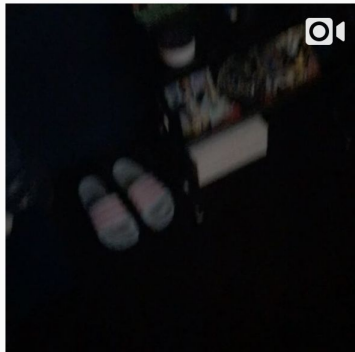
Follow

31,481 posts

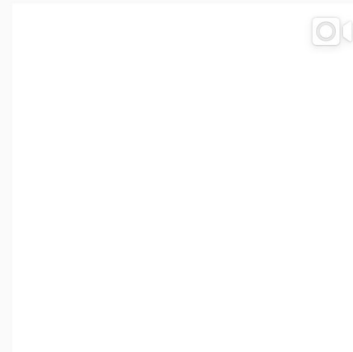
19.3m followers

2,792 following

snoopdogg new album #NevaLeft out now! youtu.be/8LygXgBypTA



lilduval



API Visual (cont.)

```
{
  "data": {
    "id": "1574083",
    "username": "snoopdogg",
    "full_name": "Snoop Dogg",
    "profile_picture": "https://scontent-iad3-1.cdninstagram.com/t51.2885-
    "bio": "snoopdogg new album #NevaLeft out now! youtu.be/8LygXgBypTA",
    "website": "http://snoopdogg.com",
    "counts": {
      "media": 31481,
      "follows": 19300000,
      "followed_by": 3410
    }
  }
}
```


API Endpoints

- Endpoints specify where resources lie
- They are really just URLs
 - You can pass data to these URLs just like any other webpage
 - i.e. `http://www.facebook.com/something?fname=Jal&lname=Irani`
- From these endpoints, normally some data is returned
 - Normally more specific data is returned if there are parameters

Endpoint Examples

- <https://developer.twitter.com/en/docs/accounts-and-users/follow-search-get-users/overview>
- <https://developer.spotify.com/web-api/endpoint-reference/>

HTTP Example

```
var http = require('http');

var server = http.createServer(function (req, res) {
  console.log('This will print in the server console');
  console.log(req.url);
  res.end('This will print on the webpage localhost:3000');
});

server.listen(3000);
```

Breakdown

- *npm install http* and *var http = require('http')* just like any other modules
- The `createServer` function will then listen on a specific port specified by the `.listen()` function
- Note: `createServer` takes a callback function that two parameters:
 - Request: The incoming request from the client
 - Response: Our response as a server to the requesting client

Breakdown (cont.)

- `res.end()` function
 - Will write to the webpage. This is part of the actual response
- `req.url` will return the url that the client navigated to
- Note we are setting the `http.createServer` equal to a server variable
- This is giving us a server instance so that we can listen on a specific port number, etc.

Obtaining Parameters from a URL

- There is a nifty module called 'url' that will help parse through a url to see if there are parameters specified in the url
- The url module has a function called 'parse' that will allow a developer to break down a url into its constituent parts

url Example

```
var http = require('http');
var url = require('url');

var server = http.createServer(function (req, res) {
  var parsedUrl = url.parse(req.url, true);
  console.log(parsedUrl);
});

server.listen(3000);
```

url Example (cont.)

- What was the URL that I specified?
- How do I obtain the parameters?

```
Url {  
  protocol: null,  
  slashes: null,  
  auth: null,  
  host: null,  
  port: null,  
  hostname: null,  
  hash: null,  
  search: '?fname=%22jal%22&lname=%22irani%22&is=%22TheMan%22',  
  query: { fname: '"jal"', lname: '"irani"', is: '"TheMan"' },  
  pathname: '/',  
  path: '/?fname=%22jal%22&lname=%22irani%22&is=%22TheMan%22',  
  href: '/?fname=%22jal%22&lname=%22irani%22&is=%22TheMan%22' }
```


url.parse() Note

- In the previous example, `parsedUrl.query.fname` would give you 'jal'

Status Codes

- We will cover this in more depth next lecture when talking about HTTP requests when we are the client
- For now the important thing to note is
- A 200 or 201 response status code means that the request was successful
- Anything in the 4xx's means that the request was not successful
- In short, always check for a 2xx status code!!!
 - Since we are a server, if the request works out well, then we should send a 2xx status code!!!

Sending a Status Code to a Requester

- People that use our API need to receive some sort of status code to know how the request that they sent was processed
- `res.writeHead()` does the trick

Url Example

- Url = `http://localhost:3000/test/endpoint?name=jalirani`

Things to note

- `JSON.stringify(object)` : converts the JSON object to String. Not completely necessary but is useful sometimes
- The 'Content-Type': 'application/json' just tells the requester that the content type being returned is JSON. Not entirely necessary but is good practice. You will see this is very common in server programming